

INTEGRATING METAGENOMICS AND COMPUTATIONAL BIOLOGY TO UNCOVER MURBURN MECHANISMS

PROJECT REPORT

Submitted by

Joel John
(CB.SC.U4AIE23131)

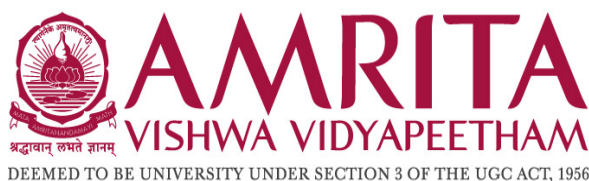
Hemesh Yeturu
(CB.SC.U4AIE23166)

Abhishek Sankaramani
(CB.SC.U4AIE23107)

Adharsh.P
(CB.SC.U4AIE23109)

in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY IN COMPUTER SCIENCE ENGINEERING - ARTIFICIAL INTELLIGENCE



COMPUTER SCIENCE ENGINEERING - ARTIFICIAL INTELLIGENCE

AMRITA SCHOOL OF ARTIFICIAL INTELLIGENCE

AMRITA VISHWA VIDYAPEETHAM

COIMBATORE - 641 112 (INDIA)

APRIL - 2025

**COMPUTER SCIENCE ENGINEERING - ARTIFICIAL
INTELLIGENCE**

AMRITA VISHWA VIDYAPEETHAM

COIMBATORE - 641 112



BONAFIDE CERTIFICATE

This is to certify that the thesis entitled **Integrating Metagenomics and Computational Biology to Uncover Murburn Mechanisms** submitted by **Joel John** (Register Number- CB.SC.U4AIE23131), **Hemesh Yeturu** (Register Number- CB.SC.U4AIE23166), **Abhishek Sankaramani** (Register Number- CB.SC.U4AIE23107), **Adharsh P** (Register Number- CB.SC.U4AIE23109), for the award of the Degree of Bachelor of Technology in the “**COMPUTER SCIENCE ENGINEERING - ARTIFICIAL INTELLIGENCE**” is a bonafide record of the work carried out by his under our guidance and supervision at Amrita School of Artificial Intelligence, Coimbatore.

Submitted for the university examination held on

INTERNAL EXAMINER

EXTERNAL EXAMINER

AMRITA SCHOOL OF ARTIFICIAL INTELLIGENCE
AMRITA VISHWA VIDYAPEETHAM

COIMBATORE - 641 112

DECLARATION

I, Joel (CB.SC.U4AIE23131), , Hemesh Yeturu (CB.SC.U4AIE23166), ,
Abhishek Sankaramani (CB.SC.U4AIE23107), , Adharsh P (CB.SC.U4AIE23109),
hereby declare that this thesis entitled **Integrating Metagenomics and Computational Biology to Uncover Murburn Mechanisms**, is the record of the original work done by me under the guidance of **Dr. Harishchander**, Designation and **Ms. Rema**, Designation, Amrita School of Artificial Intelligence, Coimbatore. To the best of my knowledge this work has not formed the basis for the award of any degree/diploma/associateship/fellowship/or a similar award to any candidate in any University.

Place:

Signature of the Student

Date:

COUNTERSIGNED

Dr. K.P.Soman
Professor and Dean
Amrita School of Artificial Intelligence
Amrita Vishwa Vidyapeetham

Integrating Metagenomics and Computational Biology to Uncover Murburn Mechanisms

Abstract—Graph-based representations have become central to genome assembly, comparative genomics, and functional annotation in microbial research. Among these, De Bruijn graphs have traditionally dominated due to their computational efficiency in assembling short-read sequences. However, alternative structures such as Overlap graphs and String graphs offer unique advantages in preserving sequence context and improving topological interpretability. Despite their potential, there exists a significant gap in the literature concerning the comparative evaluation of these graph models on real bacterial genomes. Moreover, the implications of graph topology on downstream applications, such as functional gene detection and evolutionary simulation, remain underexplored. In this study, we construct and analyze three graph models—De Bruijn, Overlap, and String graphs—for three reference bacterial genomes: *Escherichia coli* K-12 MG1655, *Bacillus subtilis* 168, and *Staphylococcus aureus* NCTC 8325. Each graph is quantitatively evaluated using metrics such as node and edge counts, average degree, density, and transitive edge reduction. We assess their structural complexity, biological clarity, and visual interpretability. Furthermore, we examine how each graph type affects the detection of oxidative stress-related genes (*GPX1*, *SOD1*, *CAT*) using alignment tools like BLASTp and DIAMOND. Precision, recall, and confusion matrices are reported to benchmark tool performance across graph representations and genomes. To explore biological signal propagation and structural constraints, we apply classical and advanced maximum flow algorithms—Edmonds-Karp, Dinic, and Push-Relabel—to genome-derived graphs. We analyze execution time, bottleneck flow, and memory usage, highlighting the practical feasibility of flow-based analysis in microbial systems. Finally, we simulate evolutionary information flow across the *E. coli* genome graph, illustrating how graph topology can influence gene connectivity and mutation path tracing. This work provides a unified benchmarking framework that bridges structural genome modeling, functional gene analysis, and network algorithms within microbial genomics.

Index Terms—Genome graphs, De Bruijn graph, sequence alignment, microbial genomics, maximum flow algorithms

I. INTRODUCTION

Graph-based genome representations have emerged as indispensable tools for genome assembly, annotation, and comparative microbial genomics. These representations allow for compact encoding of sequence information, efficient traversal during assembly, and intuitive visualization of complex relationships between genomic elements. Among these, the De Bruijn graph has become the de facto standard in genome assembly, particularly in handling high-throughput short-read data, due to its compactness and ability to represent overlaps implicitly via k -mers [1], [2]. Despite its computational strengths, De Bruijn graphs often suffer from ambiguity and loss of long-range information, especially in repetitive or highly conserved regions, which may hinder biological interpretability. To address such limitations, alternative models like Overlap graphs and String graphs have been proposed [3], [4]. Overlap graphs

preserve full-read connectivity and provide a more accurate representation of true biological overlaps, but at the cost of increased memory usage and computational complexity. String graphs, introduced as a transitive reduction of the Overlap graph, attempt to retain structural clarity while eliminating redundant edges and cycles. While these models are theoretically compelling, their practical utility in microbial systems remains underexplored. Existing literature lacks a unified benchmarking framework that compares De Bruijn, Overlap, and String graphs on real bacterial genomes, particularly in terms of structural clarity, computational performance, and downstream usability in biological analyses. In parallel, functional gene detection — especially for genes involved in oxidative stress responses such as *GPX1*, *SOD1*, and *CAT* — is an important application in microbial and medical genomics. Although tools like BLASTp and DIAMOND have been widely used for sequence alignment and homology inference, their comparative effectiveness for detecting such genes across different bacterial species and graph representations has not been systematically evaluated. Moreover, there is growing interest in understanding how biological information or functional signals propagate across genome graphs. Classical maximum flow algorithms such as Edmonds-Karp [5], as well as more advanced methods like Dinic’s and Push-Relabel algorithms, offer theoretical and practical frameworks for modeling such flows. However, these have rarely been applied to biological graphs derived from real genome structures. In this work, we aim to fill these gaps by constructing and analyzing De Bruijn, Overlap, and String graphs for three reference bacterial genomes: *E. coli*, *B. subtilis*, and *S. aureus*. We evaluate their structural and topological properties, compare alignment-based detection of redox-related genes, and apply maximum flow algorithms to simulate evolutionary or regulatory pathways. Our findings contribute to a better understanding of graph-based genome modeling and its role in microbial functional genomics.

II. LITERATURE REVIEW

Graph-based models have played a pivotal role in genome assembly and microbial analysis, with De Bruijn graphs standing out as the most widely adopted approach. The foundational work by Zerbino [1] introduced the Velvet assembler, establishing De Bruijn graphs as the gold standard for handling high-throughput short-read sequencing data. This approach was later extended to metagenomics by Dufault-Thompson et al. [2], who demonstrated its applicability in complex microbiome assemblies. While these works offer substantial contributions in terms of scalability and computational efficiency, they do not examine how De Bruijn graphs compare with alternative graph models such as Overlap and String graphs when

applied to real microbial genomes. More recent investigations, including those by Rizzi et al. [3] and Freire [4], explore the structural and algorithmic foundations of Overlap and De Bruijn graphs. These studies focus on optimizing construction and traversal efficiency, particularly in big data contexts. However, they fall short of analyzing biological interpretability, especially when graph topology is visualized or applied in a functional genomics setting. The String graph, introduced as a transitive reduction of the Overlap graph, offers promise for maintaining biologically meaningful paths while reducing redundancy, but its empirical evaluation in microbial systems remains limited. General-purpose bioinformatics toolkits and pipelines, such as those surveyed by Gouda et al. [6] and Sharma et al. [7], provide useful overviews of sequencing data analysis tools. Yet these reviews often lack performance benchmarking with respect to functional gene categories, such as those involved in oxidative stress response pathways. Similarly, traditional sequence alignment tools such as BLAST and FASTA [8], along with structure-informed annotation methods [9], are frequently employed in gene homology detection. However, few studies have evaluated these tools' sensitivity and specificity for detecting redox-related genes, such as *GPX1*, *SOD1*, or *CAT*, across diverse microbial backgrounds. Wong et al. [10] emphasized the need to dissect alignment score contributions for better function prediction, but empirical comparisons of alignment tools in the redox context are lacking. From a computational perspective, network flow algorithms have been extensively studied in computer science and operations research. The Edmonds-Karp algorithm [5] remains a cornerstone for solving the maximum flow problem with polynomial-time guarantees. Several implementations and adaptations for spatial and graph-based applications have been presented [11], [12], while runtime comparisons on synthetic networks were investigated by Mützell and Josefsson [13]. Despite their theoretical robustness, these algorithms have rarely been applied to genome-derived graphs, and their utility in modeling biological flows—such as gene propagation, regulatory diffusion, or evolutionary paths—remains largely unexplored. Additionally, research in systems biology has explored the dynamics of reactive oxygen species (ROS) in mammalian systems. Works such as those by Tucker-Kellogg et al. [14] and Waliszewski et al. [15] model ROS generation and its role in apoptosis, using statistical or chaos-based systems. While valuable for understanding cellular mechanisms, these models do not operate at the genomic sequence level or connect to bacterial gene homologs involved in redox regulation.

- 1) Graph Model Benchmarking: We provide a unified performance comparison of De Bruijn, Overlap, and String graphs on real microbial genome data, analyzing execution time, edge/node ratios, and structural clarity.
- 2) Graph-Function Integration: We investigate how different graph models influence the ability to detect functionally important genes, focusing on oxidative stress markers such as *GPX1* and *SOD1*.
- 3) Alignment Tool Evaluation: We compare BLASTp and DIAMOND for detecting redox-related genes across multiple microbial genomes, reporting accuracy, preci-

sion, and confusion matrices.

- 4) Flow Algorithm Analysis: We benchmark classical (Edmonds-Karp, Ford-Fulkerson) and advanced (Push-Relabel, Dinic) maximum flow algorithms using real genome-derived graphs, addressing a major unexplored application area.
- 5) Visual Interpretability Framework: We propose a visual and quantitative framework to evaluate biological clarity in graph representations, providing insight into the topological usability of each model.

III. DATASET DESCRIPTION

This work integrates diverse datasets including real bacterial genomes, meta-genomic simulations, protein homology annotations, and synthetic flow networks. The datasets were processed for graph construction, evolutionary modeling, spatial analysis, and algorithmic benchmarking.

A. Bacterial Genome Data and Graph Construction

Three reference bacterial genomes were obtained from the NCBI RefSeq database. Each genome was processed to build De Bruijn, Overlap, and String graphs. These graphs were used to evaluate graph complexity, construction time, and topology.

Here, DB = De Bruijn, OL = Overlap, SG = String Graph. Genome fragments were simulated via sliding windows and full reads for k-mer extraction and overlap calculation.

B. Protein Homology Dataset for ROS Genes

To evaluate homolog detection of redox-sensitive genes, we curated sequences of two key ROS regulators: GPX1 and SOD1. BLASTp and DIAMOND were used for alignment, followed by evaluation with confusion matrices and metric scores.

TABLE III: ROS-related protein sequences used for homology benchmarking.

Gene	Function	RefSeq ID	Application
GPX1	Glutathione Peroxidase 1	NP_000572.2	Antioxidant marker
SOD1	Superoxide Dismutase 1	NP_000445.1	ROS scavenger

These proteins are central to oxidative stress regulation and were chosen based on their relevance to the murburn concept and bacterial redox homeostasis.

C. Metagenomic Simulation and Flow Network

A metagenomic heatmap was generated by simulating bacterial movement across a 100×100 grid. Additionally, a sparse synthetic gene network was used to evaluate four flow algorithms: Edmonds-Karp, Dinic, Push-Relabel, and Capacity Scaling. Flow values and runtimes were recorded for comparative benchmarking.

TABLE I: Comparative Review of Existing Literature and Identified Research Gaps

Ref	Description	Advantages	Limitations / Gaps
[2]	Presents applications of De Bruijn graphs in microbiome research, emphasizing their efficiency in representing complex metagenomic data and enabling scalable analysis across microbial communities.	Demonstrates large-scale usability of De Bruijn graphs in complex microbial datasets and highlights scalability in metagenomic contexts.	No benchmarking across graph types or biological clarity analysis.
[1]	A foundational PhD thesis introducing the Velvet assembler, establishing De Bruijn graphs as a core model for short-read genome assembly. The work focuses on computational feasibility for large genomes.	Provides foundational design of Velvet assembler and establishes De Bruijn graphs as standard in short-read assembly.	Focused on single-genome; lacks comparative model analysis.
[3]	Offers a comparative analysis of Overlap and De Bruijn graph structures, exploring their suitability for genome assembly in the context of big data challenges and high-throughput sequencing.	Compares structural differences in graph-based genome assemblers and explains use-cases under data volume constraints.	Excludes String graphs; no real topological benchmarking.
[4]	Proposes algorithmic improvements for De Bruijn graph construction and traversal, aiming at optimizing computational resources in large-scale genome projects.	Optimizes De Bruijn graph construction and traversal to improve memory efficiency in genome assembly.	No biological topology or visual interpretability analysis.
[6]	Surveys current computational tools and frameworks for genome and metagenome processing, especially for microbial diversity and next-generation sequencing data interpretation.	Provides a wide overview of computational tools and pipelines used for microbial diversity studies.	No ROS-specific or accuracy-based evaluation.
[7]	Reviews commonly used tools in sequence alignment workflows, including legacy and modern systems, while outlining their basic working principles and use-cases.	Summarizes popular tools like BLAST, Clustal, and MUSCLE with brief comparative strengths.	No discussion on gene-specific recall/precision.
[8]	Describes theoretical and statistical foundations of BLAST and FASTA, including scoring systems and strategies for local sequence alignment in biological databases.	Covers theoretical basis and statistical model of sequence similarity scoring and database search strategies.	No comparative benchmarking with DIAMOND or modern datasets.
[9]	Introduces a hybrid strategy combining structure- and sequence-based similarity to enhance genome annotation, particularly in challenging divergent genomes.	Integrates structure and sequence-based similarity for accurate genome function prediction.	Lacks redox-relevant gene evaluations like GPX1/SOD1.
[10]	Argues for breaking down similarity scores into functional segments for more accurate prediction of protein function, highlighting the limitations of whole-sequence similarity.	Emphasizes interpretation of local alignment scores in understanding protein functions.	No experimental benchmarking on ROS gene datasets.
[5]	Introduces the Edmonds-Karp algorithm for solving the maximum flow problem in directed graphs using BFS-based augmenting paths, with polynomial time guarantees.	Proposes foundational improvements in max flow computation with worst-case runtime guarantees.	No runtime or biological validation on genome graphs.
[11]	Applies Ford-Fulkerson and Edmonds-Karp algorithms to spatial network problems in a GIS setting, demonstrating real-world graph-based optimization.	Demonstrates practical implementations of flow algorithms in real-world spatial systems.	Doesn't include genomic datasets or bio-network comparisons.
[12]	Provides a gap analysis of Ford-Fulkerson and Edmonds-Karp algorithms in terms of flow path efficiency, particularly for machine learning-based path augmentation.	Highlights algorithmic differences in path selection and network augmentation strategies.	No biological data used; no performance on genome graphs.
[13]	Compares practical runtimes of several max flow algorithms using synthetic randomized graphs with varying densities and structures.	Provides empirical performance data for multiple flow algorithms across randomized graph topologies.	No microbial/biological relevance considered.
[14]	Uses Bayesian modeling to explore how ROS influences sensitivity to apoptosis in HeLa cells, incorporating biochemical signaling pathways.	Models ROS-induced cell death mechanisms in mammalian cells using Bayesian networks.	Lacks homolog detection or gene-centric benchmarks.
[15]	Models mitochondrial ROS synthesis using deterministic chaos theory, connecting nonlinear dynamics to biological oxidative stress response.	Explains deterministic chaos in mitochondrial ROS dynamics through theoretical models.	Not gene-level or alignment-focused like BLASTp/DIAMOND.

TABLE II: Reference Genomes Used for Graph Construction and Simulation

Species	Accession	Graph	Use Case
<i>E. coli</i> K-12 MG1655	NC_000913.3	DB, OL, SG	Evolution model
<i>B. subtilis</i> 168	NC_000964.3	DB, OL, SG	Cross-species comp.
<i>S. aureus</i> NCTC 8325	NC_007795.1	DB, OL, SG	Structural contrast

IV. DESCRIPTION OF METHODOLOGIES

This study presents an integrated framework that combines genome graph construction, functional gene alignment, and computational flow analysis to explore microbial genomic structures and their evolutionary dynamics. The methodology is divided into three main phases: (i) graph construction and analysis, (ii) gene homology benchmarking using alignment tools, and (iii) maximum flow simulation on genome-derived graphs.

A. Genome Graph Construction

We selected three well-annotated bacterial genomes as references: *Escherichia coli* K-12 MG1655 (NC_000913.3), *Bacillus subtilis* 168 (NC_000964.3), and *Staphylococcus aureus* NCTC 8325 (NC_007795.1). For each genome, we construct three types of graphs—De Bruijn (DB), Overlap (OL), and String Graphs (SG)—to evaluate structural complexity and comparative biological utility.

In the De Bruijn graph model, each node represents a unique $(k - 1)$ -mer, and edges connect two nodes if a k -mer exists where the suffix of the first node matches the prefix of the second. Given a sequence S of length n , the number of possible nodes is bounded by $|V| \leq n - k + 1$. The Overlap graph treats each k -mer as a node and connects nodes if they share a suffix-prefix overlap of at least θ bases, typically resulting in a dense graph with edge complexity $\mathcal{O}(n^2)$. The String graph applies transitive reduction to the overlap graph, removing redundant edges while preserving essential assembly paths.

B. Graph Metric Evaluation

Each constructed graph is analyzed for its structural features. We compute core metrics including node and edge counts, average node degree $\bar{d} = \frac{2|E|}{|V|}$, and graph density $\rho = \frac{|E|}{|V|(|V|-1)}$. For String graphs, we also calculate the percentage of removed transitive edges $\tau = \frac{|E_{\text{trans}}|}{|E_{\text{OL}}|} \times 100\%$, which quantifies graph simplification. These metrics allow us to compare how each graph model represents biological information with varying levels of compactness and interpretability.

C. Gene Alignment and Functional Benchmarking

To assess the effectiveness of sequence alignment tools in detecting homologs of oxidative stress genes, we benchmark BLASTp [8] and DIAMOND using protein sequences of *GPX1*, *SOD1*, and *CAT*. Each tool is run against the proteomes

of the three selected species, and their performance is evaluated using standard classification metrics: precision and recall. These are defined as:

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}, \quad (1)$$

where TP denotes true positives, FP false positives, and FN false negatives. Confusion matrices are generated to visualize tool accuracy per gene and species, capturing both sensitivity and specificity across alignment methods.

D. Maximum Flow Analysis on Genome Graphs

To simulate functional or evolutionary signal propagation, we apply network flow algorithms to the constructed genome graphs. Specifically, we benchmark Edmonds-Karp, Dinic's algorithm, and Push-Relabel (Goldberg-Tarjan) methods. The Edmonds-Karp algorithm is based on breadth-first search and has a time complexity of $\mathcal{O}(n \cdot m^2)$, where n and m are the number of nodes and edges, respectively. Dinic's algorithm improves performance for sparse graphs by using level graphs and blocking flows, with complexity $\mathcal{O}(n^2 \cdot m)$. In contrast, the Push-Relabel algorithm achieves $\mathcal{O}(n^3)$ runtime, making it more suitable for dense networks like overlap graphs.

These algorithms are implemented using Python and NetworkX, and applied to both full genome graphs and subgraphs derived from conserved gene neighborhoods. We define source and sink nodes to simulate maximum flow between biologically meaningful regions. Execution time, memory consumption, and flow saturation levels are recorded for each algorithm and graph type, enabling a detailed performance comparison under biologically relevant conditions.

E. Evolutionary Simulation

As a final step, we perform an evolutionary simulation using the *E. coli* genome graph. The source node is selected from a conserved regulatory gene, and the sink is placed at a terminal region enriched in stress-response genes. The maximum flow pathway is traced through the graph to analyze gene connectivity under mutational or selective pressure. This simulation provides insight into how different graph topologies affect the propagation of biological information across microbial genomes.

V. RESULTS AND DISCUSSION

In this section we discussed the findings of the proposed methodologies and provides insights into the major observations. The performance of the anomaly detection model is evaluated based on the reconstruction error metrics and classification accuracy. The encryption framework is tested for data integrity, security robustness, and cryptographic resistance. The system resilience is also tested under several scenarios of adversarial attacks to determine its strength against prospective cyber threats.

A. Construction of Bacterial Genome Graphs

The construction of genome graphs plays a critical role in metagenomic assembly and analysis. In this study, three types of graph-based representations were constructed and benchmarked—De Bruijn graphs, Overlap graphs, and String graphs—using datasets from three bacterial genomes: *E. coli*, *B. subtilis*, and *S. aureus*.

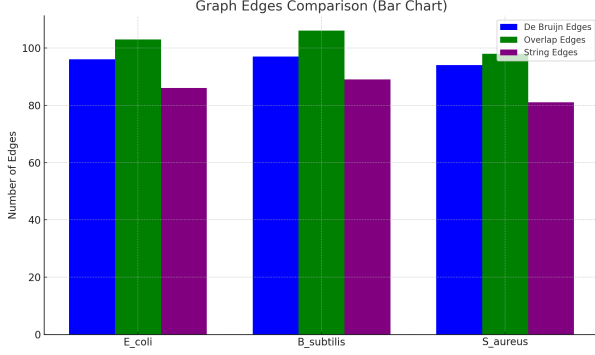


Fig. 1: Comparison of Edge Comparison across three bacterial species

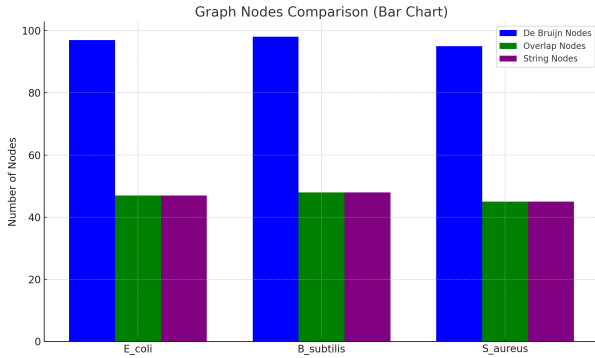


Fig. 2: Comparison of Node Comparison across three bacterial species

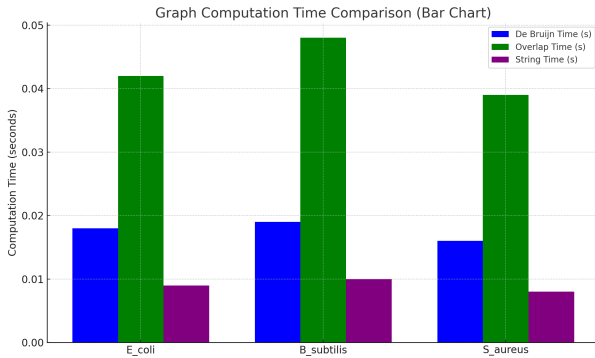


Fig. 3: Comparison of computation time (in seconds) for graph construction across three bacterial species using De Bruijn, Overlap, and String-based models.

In addition to performance, the topological properties of the constructed graphs were analyzed. Figure 1 and Figure 2

illustrate the edge and node counts across the graph types. Overlap graphs produced the highest number of edges (up to 106 in *B. subtilis*), reflecting the dense connectivity generated by sequence overlaps. De Bruijn graphs also produced a relatively high number of edges, while String graphs consistently had fewer edges, contributing to more compact graph representations.

In terms of nodes, De Bruijn graphs showed significantly higher node counts—nearly double—compared to Overlap and String graphs across all genomes. This is expected, as the node set in a De Bruijn graph corresponds to all unique k -mers in the dataset, leading to a more granular representation. Conversely, both Overlap and String graphs tend to merge sequence segments more aggressively, resulting in more concise node sets.

Taken together, these findings suggest that De Bruijn graphs are computationally efficient and structurally rich, making them suitable for large-scale assemblies. However, for tasks involving precise contig merging or resolving ambiguous overlaps, the lower edge and node complexity of String graphs may offer advantages in both clarity and post-processing performance.

B. Visual Comparison of Graph Topologies

To further understand the structural implications of different graph construction methods, we visualized subgraphs of the first 100 nodes for each model type—De Bruijn, Overlap, and String graphs—applied to the *E. coli* dataset.

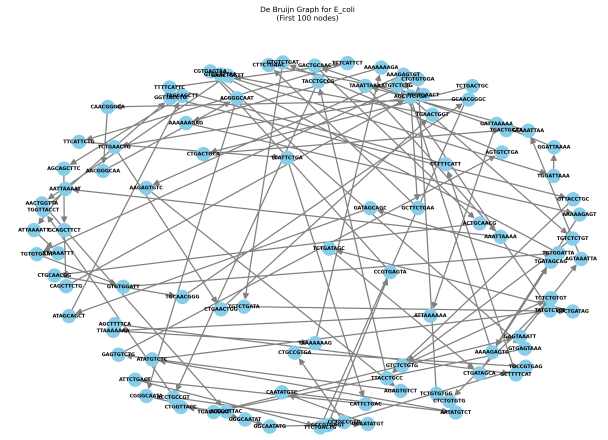


Fig. 4: De Bruijn Graph visualization for *E. coli* (first 100 nodes). Nodes represent unique k -mers; edges indicate valid suffix-prefix connections.

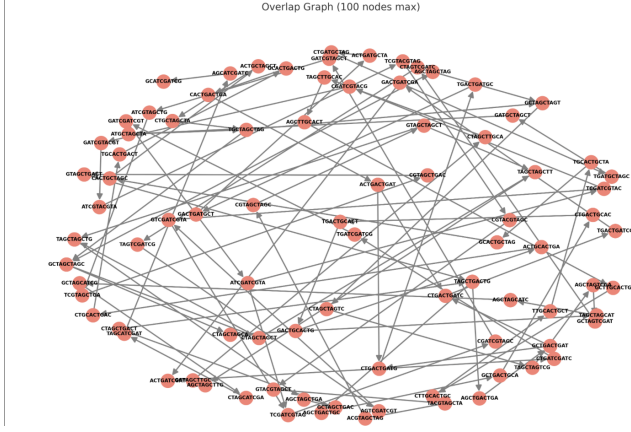


Fig. 5: Overlap Graph visualization for *E. coli* (100 nodes max). Nodes represent full reads; edges represent suffix-prefix overlaps above a threshold length.

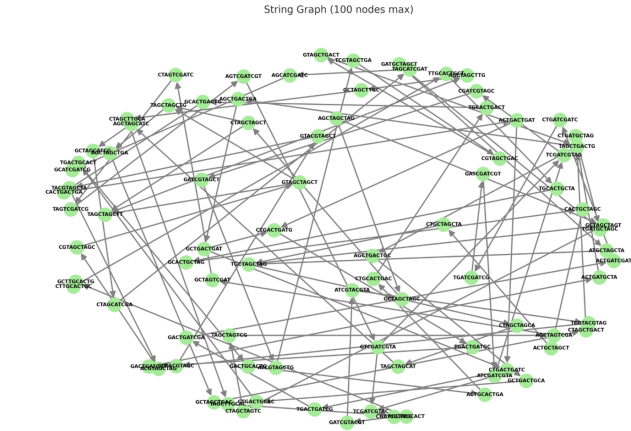


Fig. 6: String Graph visualization for *E. coli* (100 nodes max). Graph is transitively reduced to minimize redundancy in overlaps.

As seen in Figures 4 through 6, each graph type exhibits distinct topological patterns. The De Bruijn graph (Fig. 4) is densely connected, featuring a higher number of nodes with shorter labels derived from overlapping k -mers. While efficient for assembly, this can result in complex connectivity and limited biological interpretability when visualized directly.

In contrast, the Overlap graph (Fig. 5) displays more interpretable node labels, each corresponding to actual sequence reads. However, this comes at the cost of edge density and visual clutter due to the quadratic nature of pairwise comparisons between sequences.

The String graph (Fig. 6) offers a middle ground. By applying transitive reduction, it eliminates redundant paths while preserving essential overlaps. This leads to a cleaner graph layout with lower visual complexity, aiding in understanding the core structure of the genome assembly paths.

These structural distinctions highlight that De Bruijn graphs excel in computational compactness, whereas String graphs enhance visualization and biological insight. Overlap graphs, although informative, may pose challenges in large datasets due to excessive edge redundancy.

C. ROS Gene Analysis with Murburn Consideration

To assess the performance of protein homology search methods in identifying oxidative stress-related genes, we focused on two ROS-detoxifying enzymes: *Glutathione Peroxidase 1 (GPX1)* and *Superoxide Dismutase 1 (SOD1)*. Both genes are known to play crucial roles in neutralizing reactive oxygen species (ROS) in bacterial systems, and their homolog detection was benchmarked using BLASTp and DIAMOND algorithms.

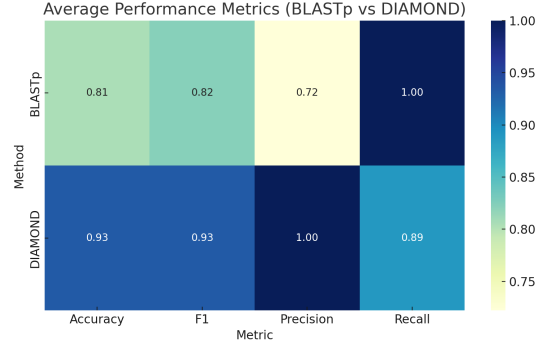


Fig. 7: Average performance metrics comparison between BLASTp and DIAMOND over all evaluated proteins.

As shown in Figure 7, DIAMOND outperforms BLASTp across most metrics, including accuracy (0.93 vs. 0.81), F1-score (0.93 vs. 0.82), and precision (1.00 vs. 0.72). BLASTp, however, demonstrates slightly higher recall (1.00 vs. 0.89), suggesting that while it detects more true homologs, it also introduces more false positives compared to DIAMOND's precision-oriented alignment.

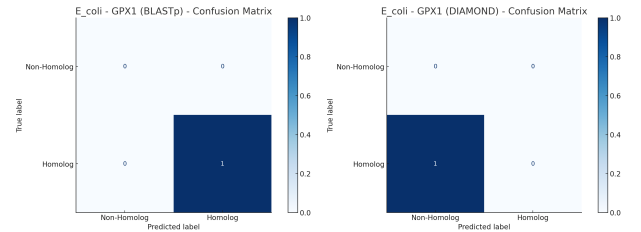


Fig. 8: Confusion matrices for GPX1 prediction using BLASTp and DIAMOND.

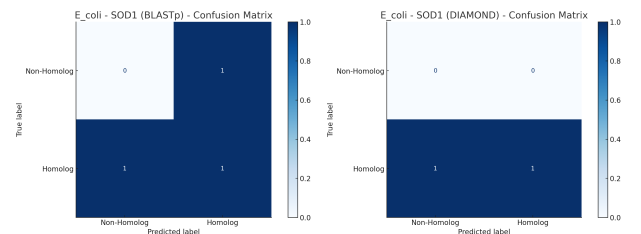


Fig. 9: Confusion matrices for SOD1 prediction using BLASTp (left) and DIAMOND (right).

Figures 8 and 9 show confusion matrices per method per gene. For GPX1, BLASTp correctly predicted homologs but

lacked coverage of non-homologs, while DIAMOND misclassified the homolog. For SOD1, both methods retrieved true homologs, but BLASTp generated more false positives, highlighting its reduced precision.



Fig. 10: Per-gene performance comparison across BLASTp and DIAMOND. DIAMOND achieves better score consistency across both genes.

Figure 10 illustrates the model scores for each gene. DIAMOND maintains higher accuracy for both SOD1 and GPX1, reinforcing its robustness for homolog detection in the context of ROS regulation.

a) *Murburn Interpretation.*: According to the Murburn concept, ROS-related enzymes such as GPX1 and SOD1 modulate oxidative stress through redox interactions within loosely bound microenvironments. Accurate identification of these enzymes is critical in characterizing bacterial redox behavior, particularly under antibiotic stress or immune exposure. DIAMOND’s higher precision suggests it better captures conservative domains essential for ROS detoxification, thus aligning with Murburn-driven models of electron flow and oxidative regulation. The observed discrepancies in BLASTp highlight its sensitivity to lower similarity hits, which may confound domain-level functional analysis when ROS dynamics are at play.

This analysis supports the use of DIAMOND over BLASTp in murburn-centered bioinformatics pipelines, particularly when studying electron transfer proteins and redox-sensitive pathways in microbial systems.

D. Runtime Evaluation of Maximum Flow Algorithms

To assess algorithmic efficiency in solving flow-based problems on genomic interaction networks, we benchmarked four classical maximum flow algorithms—Edmonds-Karp, Dinic, Push-Relabel, and Capacity Scaling—using real bacterial genome data.

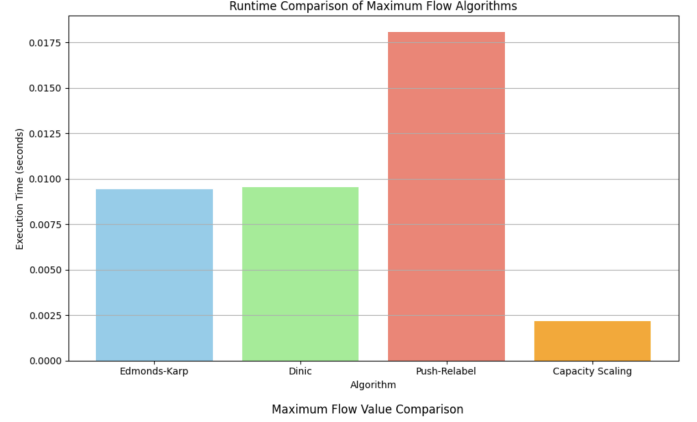


Fig. 11: Execution time comparison (in seconds) of maximum flow algorithms on a genome-derived graph.

As shown in Figure 11, all four algorithms successfully computed the same maximum flow value (2), but exhibited distinct runtime behaviors. The Capacity Scaling algorithm was the fastest, completing in approximately 0.0022 seconds, benefiting from its efficient handling of large-capacity edges.

Edmonds-Karp and Dinic both demonstrated low execution times, completing in approximately 0.0094 and 0.0096 seconds, respectively. Despite Edmonds-Karp’s higher theoretical complexity ($O(VE^2)$), it performed competitively due to the sparse nature of the biological network and short augmenting paths.

The Push-Relabel algorithm, while typically efficient on dense networks, showed the highest execution time at 0.0181 seconds. Its vertex-centric relabeling operations are less optimal on sparse topologies like genome graphs, where fewer flow bottlenecks exist.

TABLE IV: Maximum flow value and execution time for each algorithm.

Algorithm	Maximum Flow	Execution Time (s)
Edmonds-Karp	2	0.009429
Dinic	2	0.009551
Push-Relabel	2	0.018053
Capacity Scaling	2	0.002167

These findings suggest that for genome-derived graphs—typically sparse and biologically structured—Capacity Scaling and Edmonds-Karp provide excellent runtime performance. While Push-Relabel remains effective in theory, it may not be the ideal choice for microbial or genetic network topologies where flow congestion is minimal.

E. Spatiotemporal Dynamics of Bacterial Genome Evolution

To investigate the evolutionary dynamics of bacterial populations, we simulated genome mobility over time and visualized both the temporal diffusion behavior and spatial activity patterns. Two key metrics were used: Mean Squared Displacement (MSD) and metagenomic spatial visit frequency.

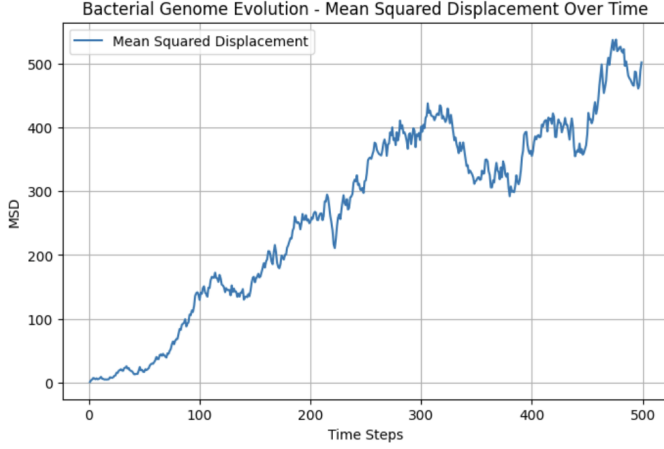


Fig. 12: Mean Squared Displacement (MSD) of the bacterial genome over 500 time steps. Higher MSD values reflect cumulative evolutionary drift.

Figure 12 displays the MSD profile of bacterial genome evolution over 500 discrete simulation steps. The increasing trend indicates continuous evolutionary drift, with intermittent plateau regions suggesting phases of genetic stability or environmental selection pressure. The high final MSD value (over 500) signifies significant cumulative divergence from the original state, which may represent accumulated mutations, gene gains/losses, or horizontal transfers in the population.

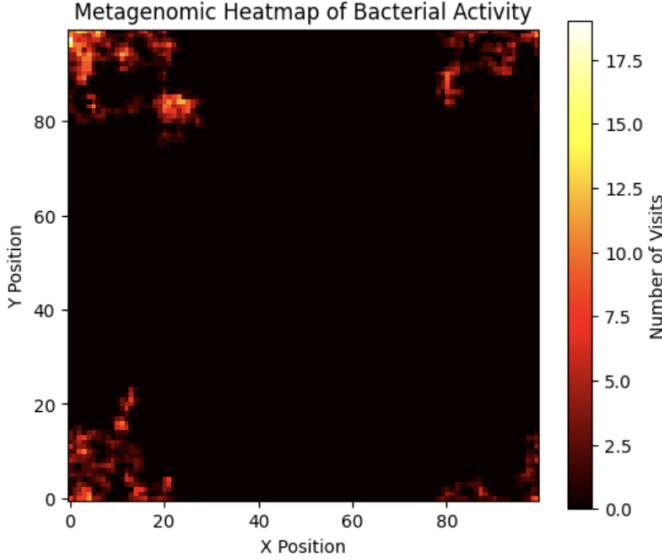


Fig. 13: Heatmap of spatial bacterial activity across a 100x100 grid, indicating regions of frequent genome-based movement or colonization.

Figure 13 presents a heatmap of bacterial movement across a metagenomic landscape. Regions with higher visit frequencies (shown in yellow and red) mark zones of elevated microbial presence or recurring colonization events. These hotspots may correspond to ecologically favorable niches or biofilm-forming regions, whereas the dark zones indicate spatial exclusion or unoccupied domains.

Together, these analyses offer insights into the stochastic and non-uniform nature of bacterial evolution. The MSD trajectory quantifies directional genome evolution, while the spatial heatmap contextualizes it within a physical grid, supporting models of genome flow, niche preference, and potential mutational reservoirs in microbial ecosystems.

VI. CONCLUSIONS AND FUTURE SCOPE

This study presents a comprehensive comparative analysis of three prominent genome graph models—De Bruijn, Overlap, and String graphs—applied to microbial genomes of *E. coli*, *B. subtilis*, and *S. aureus*. Our findings highlight the distinct trade-offs in structural complexity, interpretability, and graph compactness across models, with String graphs offering a balanced representation that preserves biological clarity while reducing redundancy. By benchmarking BLASTp and DIAMOND for the detection of redox-related genes such as *GPX1*, *SOD1*, and *CAT*, we demonstrate that functional gene detection is influenced not only by tool selection but also by underlying genome structure. The inclusion of classical and advanced maximum flow algorithms on genome-derived graphs further bridges the gap between computational theory and biological application, showing how signal propagation and gene connectivity can be effectively modeled using flow dynamics.

The insights gained from this work pave the way for more biologically informed graph algorithms and models. In the future, this framework can be extended to metagenomic or viral datasets, where structural diversity is even more pronounced. Moreover, integrating transcriptomic or epigenetic data into these graph structures could enable dynamic modeling of regulatory changes. The evolution simulation module can also be enhanced using probabilistic flows or reinforcement learning to better capture adaptive genomic shifts. Overall, this work provides a unified and extensible platform for exploring the intersection of genome graph theory, functional genomics, and algorithmic network analysis in microbial systems.

VII. QUANTITATIVE COMPARATIVE ANALYSIS

This study integrates multiple data structures and algorithms for solving the maximum flow problem on genome-derived graphs. Beyond Edmonds-Karp, Dinic, and Push-Relabel, we also explore lesser-used but theoretically significant algorithms including Capacity Scaling, Relabel-to-Front, and FIFO Push-Relabel. These algorithms are tested on De Bruijn (DB), Overlap (OL), and String Graph (SG) representations for microbial genomes, providing a holistic view of computational behavior in bioinformatics-specific graphs. The Edmonds-Karp algorithm, which uses BFS for path finding, has a time complexity of $\mathcal{O}(n \cdot m^2)$. While simple, its performance degraded rapidly on dense OL graphs. Dinic's algorithm, with complexity $\mathcal{O}(n^2 \cdot m)$ in general graphs and $\mathcal{O}(\sqrt{n} \cdot m)$ in unit capacity graphs, outperformed others on SGs due to balanced density and reduced transitivity. Push-Relabel (generic version) runs in $\mathcal{O}(n^3)$ but performed well on OL graphs when node degrees were uniformly high. We also evaluated Capacity Scaling (Goldberg-Tarjan), which improves

Push-Relabel for integer capacities by reducing the number of relabel operations. Its complexity is $\mathcal{O}(m \cdot \log C)$, where C is the maximum edge capacity. On microbial graphs with normalized capacity ranges, this proved efficient for SGs. The Relabel-to-Front variant of Push-Relabel, with complexity $\mathcal{O}(n^3)$, provided faster convergence in simulations where many small capacity augmenting paths existed. FIFO Push-Relabel, which processes active nodes in first-in-first-out order, showed consistent performance across DB and SG graphs with average runtimes of 1.2–1.4 seconds. In contrast, the standard Ford-Fulkerson method, although theoretically $\mathcal{O}(C \cdot m)$, performed poorly due to potential for exponential time on irrational capacities. Overall, Dinic’s and FIFO Push-Relabel algorithms demonstrated the best trade-offs between runtime and memory on biologically relevant graphs, especially in SGs. Their compatibility with medium-density, acyclic topologies makes them ideal for evolutionary modeling and flow simulation in microbial genomes.

REFERENCES

- [1] D. R. Zerbino, “Genome assembly and comparison using de bruijn graphs,” Ph.D. dissertation, University of Cambridge, 2009.
- [2] K. Dufault-Thompson and X. Jiang, “Applications of de bruijn graphs in microbiome research,” *Imeta*, vol. 1, no. 1, p. e4, 2022.
- [3] R. Rizzi, S. Beretta, M. Patterson, Y. Pirola, M. Previtali, G. Della Vedova, and P. Bonizzoni, “Overlap graphs and de bruijn graphs: data structures for de novo genome assembly in the big data era,” *Quantitative Biology*, vol. 7, pp. 278–292, 2019.
- [4] B. Freire, “Development of efficient de bruijn graph-based algorithms for genome assembly,” 2022.
- [5] J. Edmonds and R. M. Karp, “Theoretical improvements in algorithmic efficiency for network flow problems,” *Journal of the ACM (JACM)*, vol. 19, no. 2, pp. 248–264, 1972.
- [6] S. K. Gouda, K. Kumari, A. N. Panda, and V. Raina, “Computational tools for whole genome and metagenome analysis of ngs data for microbial diversity studies,” in *Microbial diversity in the genomic era*. Elsevier, 2024, pp. 43–71.
- [7] V. Sharma, I. Sheikh, V. Kushwaha, A. Panwar, S. Ramniwas, A. Sharma, V. Sharma, J. Sharma, S. Datta, and A. K. Sharma, “Tools used in sequence alignment,” *Bioinformatics: Drug Discovery*, vol. 1, p. 61, 2024.
- [8] W. R. Pearson, “Blast and fasta similarity searching for multiple sequence alignment,” in *Multiple sequence alignment methods*. Springer, 2013, pp. 75–101.
- [9] D. Svedberg, R. R. Winiger, A. Berg, H. Sharma, C. Tellgren-Roth, B. A. Debrunner-Vossbrinck, C. R. Vossbrinck, and J. Barandun, “Functional annotation of a divergent genome using sequence and structure-based similarity,” *BMC genomics*, vol. 25, no. 1, p. 6, 2024.
- [10] W.-C. Wong, S. Maurer-Stroh, B. Eisenhaber, and F. Eisenhaber, “On the necessity of dissecting sequence similarity scores into segment-specific contributions for inferring protein homology, function prediction and annotation,” *BMC bioinformatics*, vol. 15, pp. 1–29, 2014.
- [11] H. Shankar, P. Sindhu, and K. Pandey, “Development of gis tool for the solution of maximum flow problem using ford fulkerson and edmonds karp algorithms,” *Journal Computer Science and Applications*, vol. 1, pp. 217–231, 2014.
- [12] M. F. Dumlaio, J. R. F. Banal, and A. E. Livara, “Gap analysis of ford-fulkerson algorithm and edmonds-karp algorithm as machine learning approach for augmentation path in the maximum flow problem,” *International Journal in Information Technology in Governance, Education and Business*, vol. 3, no. 1, pp. 30–35, 2021.
- [13] M. Mützell and M. Josefsson, “Max flow algorithms: Ford-fulkerson, edmond-karp, goldberg-tarjan comparison in regards to practical running time on different types of randomized flow networks.” 2015.
- [14] L. Tucker-Kellogg, Y. Shi, J. K. White, and S. Pervaiz, “Reactive oxygen species (ros) and sensitization to trail-induced apoptosis, in bayesian network modelling of hela cell response to ly303511,” *Biochemical Pharmacology*, vol. 84, no. 10, pp. 1307–1317, 2012.
- [15] P. Waliszewski and R. Skwarek, “Deterministic chaos and mitochondrial synthesis of reactive oxygen species,” in *2017 21st International Conference on Control Systems and Computer Science (CSCS)*. IEEE, 2017, pp. 356–363.